Java 8 features:

1) Lambda (λ) Expressions.
2) Functional Interfaces.
3) Default methods in interfaces.
4) static methods in interfaces
5) predicate
6) function
7) consumer
} pre defined functional interfaces.
8) method Reference & constructor Reference
   By double colon (::) operator.
9) Stream API
10) Date & time API
    (Joda API)
    joda.org

The main purpose of java 1.8 v is:-

* To simplify the programming.
* To utility of functional interface benefits in java.
* To enable parallel programming
*

1. Lambda Expression:-

* To enable functional programming benefits in java
* write more readable, maintainable & concise code
* To use APIs very easily & effectively
* To enable parallel processing.

why Lambda Expression in java:-

Same

* The term Lambda is a mathematics word it is very helpful in MATHEMATICS further we are using programming Language.
* The first language is used Lambda is Lisp after C, C++, python ..... etc and lately the lambda concept came into java picture.

Lambda Expression :-

Lambda expression is nothing but anonymous functions means function without having name, returntype and modifiers such type of function is said to be Lambda expression.

Syntax :- is anonymous fn
- Not having name
- Not having modifiers
- Not having returntype

eg :- method to Lambda Expression

eg1

```
public void m()
{
 s.o.pln("Hello");
}
```

```
() -> {s.o.pln("Hello");}
```

eg2 :-

```
public void m(int a, int b)
{
 s.o.pln(a+b);
}
```

```
(int a, int b) -> {s.o.pln(a+b);}
```

eg3 :-

```
public int getLength(String s){
 return s.Length();
}
```

```
(String s) -> { return s.length(); }
```

eg 1:- public void m1(){
S.o.println("Hello");
}

⇒ ()->{S.o.println("Hello");}

⇓

()->S.o.println("Hello");

eg 2:- public void m1(int a, int b)
{
S.o.println(a+b);
}

⇒ (int a, int b)->{S.o.println(a+b);}

⇓

(a,b)->S.o.println(a+b);

eg 3:- Public int length(String s){
return s.length();
}

⇒ (String s)->{return s.length();}

⇓

(String s)->return s.length();

⇓

(s)->return s.length();

⇓

(s)->s.length();

⇓

s->s.length();

Conclusion :-

* body with one statement curly braces are optional.
* Type inference is optional in formal arguments

eg:- (int a, int b)->S.o.println(a+b);

⇓

(a,b)->S.o.println(a+b);

eg 1:- public void m1(){
s.o.pln("Hello");
}

⇒ ()->{s.o.pln("Hello");}

⇓

()-> s.o.pln("Hello");

eg 2:- public void m1(int a, int b)
{
s.o.pln(a+b);
}

⇒ (int a, int b)->{s.o.pln(a+b);}

⇓

(a, b)-> s.o.pln(a+b);

eg 3:- public int length(String s){
return s.length();
}

⇒ (String s)->{return s.length();}

⇓

(String s) -> return s.length();

⇓

(s) -> return s.length();

⇓

(s) -> s.length();

⇓

s -> s.length();

Conclusion:-

* body with one statement curly braces are optinal.
* Type inference is optional in formal arguments

eg:- (int a, int b) -> s.o.pln(a+b);

⇓

(a,b) -> s.o.pln(a+b);

* return type is also optional.
* in formal arguments only argument these means parantesis also optional.

eg:- (s) → s.length();
⇓
s → s.length;

Characterstics / properties of Lambda expression:-

1). A λ-expression can take any no of parameters

```
() → s.o.pln ("Hello");
(s) → s.length();
(a,b) → s.o.pln (a+b);
```

2) if multiple parameters present then the parameters should be represent with ✓

(a,b) → s.o.pln (a+b);

3) if only one parameter available then parantesis are optional

(s) → s.length(); ⟹ s → s.length();

4) usually we can specify the type of parameter of compiler expect the type based on context, then we can remove type [Type reference].

(int a, int b) → s.o.pln (a+b);
⇓
(a,b) ⟹ s.o.pln (a+b);

5) Similar to method body, λ-expression body can contain any number of stmts of multiple stmts are there then we should enclose within any braces.

eg:- () → { stmt1;
stm 2;
stm 3;
}

If body containing only one statement then any braces are optional.

eg:- () → s.println("Hello");

6) if λ-expression return something then we can remove return & keyword.

(s) s → return s.length();

⇓

s → s.length();

Functional Interface:-

* A function which contain single abstract method such type of interfaces is called functional interface

* we can have default methods, static methods any number of times.

eg:-

```
@functionalInteface
Interface Inter{
    public void m1();
    default void m2(){
    }
    static void m3(){
    }
}
```

✓

```
@functionInterface{
Interface inter{
    public void m1();
    public void m2();
}.
```

✗

eg:- callable, runnable, comparable have only one abstract method

@ Functional Interface Annotation:-

Functional interface w.r.t to inheritance:-

Case 1:-

* an interface extends functional interface and child interface does not contain any abstract method, then child interface is always functional interface.

eg:-

```
@FunctionalInterface
interface P {
    public void m1();
}
@FunctionalInterface
interface C extends P {

}
```

✓

case 2:-

in the child interface, we can define exactly same parent interface abstract method.

eg:-

```
@FunctionalInterface
interface P {
    public void m1();
}
@FunctionalInterface
interface C extends P {
    public void m1();
}
```

✓

case 3:-

in the child interface we can't define any new abstract methods otherwise we will get CE.

eg:-

```
@FunctionalInterface
interface P {
    public void m1();
}
@FunctionalInterface
interface C extends P {
    public void m2();
}
```

✗

CE: unexpected @FunctionInterface Annotation
multiple non-overriding abstract methods

case 4:-

```
@Functional interface
interface P {
    public void m1();
}
interface C extends P {
    public void m2();
```

✓

Invoking Lambda expression by using Function Interface :

ex1 : without λ-expression :-

```
interface Interf {
    public void m1();
}
class Demo implements Interf {
    public void m1() {
        S.o.pln("normal implementation");
    }
}
class Test {
    p.s v m(S[] args) {
        Interf i = new Demo();
        i.m1();
    }
}
```

with λ-expression :-

```
interface Interf {
    public void m1();
}
class Test {
    p s v m (S[] args) {
        Interf i = () -> S.o.pln("Lambda Expression implementation");
        i.m1();
    }
}
```

ex 1:-

without λ - expression :-

```
interface Intef {
    public void m1(int a, int b);
}
class Demo implements Intef {
    public void m1(int a, int b) {
        S.o.pln("The sum is:" + (a+b));
    }
}
class Test {
    mm {
        Intef i = new Demo();
        i.m1(10, 20); // 30
        i.m1(100, 200); // 300
    }
}
```

with λ - expression :-

```
interface Intef {
    public void m1(int a, int b);
}
class Demo {
    mm {
        Intef i = (a, b) -> S.o.pln("The sum is:"+(a+b));
        i.m1(10, 20); // 30
        i.m1(100, 200); // 300
    }
}
```

ex3: without λ - expression:-

```
interface Inter {
    public int getLength (string s);
}
class Demo implements Inter {
    public int getLength (String s) {
        return s.length();
    }
}
class Demo {
    mm {
        Inter i = new Demo ();
s.o.pln (i.getLength ("Hello"));
s.o.pln (i.getLength ("without lambda expression"));
}
}
```

with lambda expression:-

```
interface Inter {
    public String getLength (String s);
}
class Demo {
    mm {
        Inter i = (String s) -> s.length();
s.o.pln(i.getLength ("Bye"));
s.o.pln(i.getLength ("with λ expression"));
    }
}
```

ex4: without Lambda expressions:

```
class Demo implements Runnable {
    public void run() {
        for (int i=0; i<10; i++) {
            S.o.p("child Thread");
        }
    }
}
class ThreadsDemo {
    mm {
        Runnable r = new Runnable();
        Thread t = new Thread(r);
        t.start();
        for (int i=0; i<10; i++) {
            S.o.p("main thread");
        }
    }
}
```

defining a thread

responsible to execute child Thread

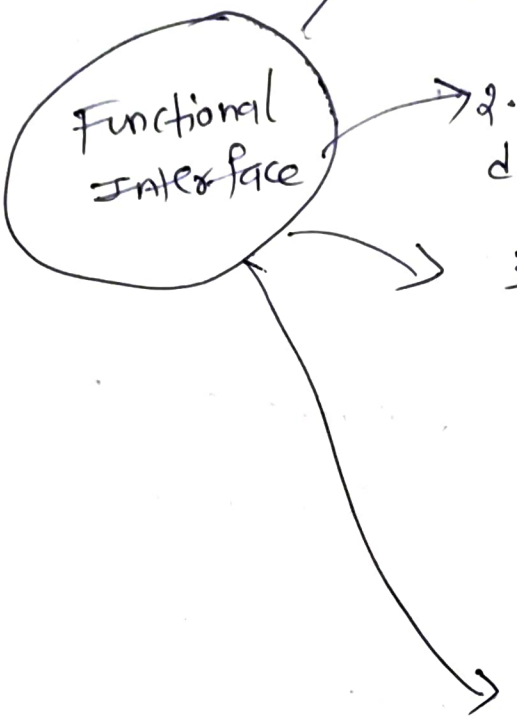
main

child main

responsible to execute main thread

with Lambda expression:

```
class ThreadDemo {
    mm {
        Runnable r = () -> { for(int i=0; i<10; i++){
                S.o.p("child. thread"); } };
        Thread t = new Thread(r);
        t.start();
        for (int i=0; i<10; i++) {
            S.o.p("main thread");
        }
    }
}
```



1. it should contains exactly one abstract method (SAM: single abstract method).

2. it can contains any number of default and static methods

3. it acts as a type for λ-expression.

eg:- Interf i = () → s.o.println ("Hello");

Interf → Type

() → ... → λ Expression

4. it can be used to invoke λ-expressions

eg:- i.m1()

ex:- Case 1:- Why Functional Interface should contains only one abstract method ?

*1)

```
interface Interf {
    public void m1(int i);
}
Interf I = (i) -> s.o.println(i*i);
I.m1(10);
I.m1(20);
```

2)

```
interface Intef {
    public void m1(int i);
    public void m2(int i);
}
```

3

```
Intef I = i -> s.o.println(i*i);
```

CE: Incompatible types Intef is not a f^n interface
multiple non-overriding abstract methods in interface
Intef

ex case 2:— what is the advantage of functional interface

```
@FunctionalInterface
interface Intef {
    public void m1();
}
```

3

```
Intef f = () -> s.o.println("Hello") || Intef i = () ->
                                        s.o.println("guru");
```

The main use is, the any Programmar is know to is mainly used for λ-expressions. if we are trying to add another abstract method means directly it shows CE: Incompatible

case 4:- our custom class Lambda expression

```
package Lambda;
import java.util.*;
class Employee {
    int empno;
    String ename;
    Employee (int empno, String ename) {
        this.empno = empno;
        this.ename = ename;
    }
    public String toString() {
        return empno + " " + ename;
    }
}
class LambdaComparable {
    psvm(String[] args) {
    ArrayList<Employee> a = new ArrayList<Employee>();
    a.add(new Employee(2, "prasad"));
    :
    a.add(new Employee(4, "ashwin"));
    s.o.println(a);
Collections.sort(a, (e1, e2) -> e1.ename.compareTo(e2.ename));
  s.o.println(a);
Collections.sort(a, (e1, e2) -> Integer.compare(e1.empno, e2.empno));
  s.o.println(a);
Collections.sort(a, (e1, e2) -> (e1.empno < e2.empno) ? -1 :
                                (e1.empno > e2.empno) ? 1 : 0));
  s.o.println(a);
```

What is Anonymus inner class :-

* An anonymous inner class is a class without a name that is declared and initialized at the same time.

* it is mainly used when you need to override a method only once and don't want to create a seperate class

Syntax :-

```
className obj = new className() {
        // method implementation.
    };
```

extending a class :-

eg1

```
class Animal {
    void sound() {
        S.o.p ("Animal Sound");
    }
}
class main {
    p s v m (S [] args) {
        Animal a = new Animal() {
            void sound() {
                S.o.p ("Dog Barks");
```

What is Anonymus inner class :-

* An anonymous inner class is a class without a name that is declared and initialized at the same time.

* it is mainly used when you need to override a method only once and don't want to create a seperate class

Syntax :-

```
className obj = new className() {
        // method implementation.
};
```

extending a class :-

eg1

```
class Animal {
    void sound() {
        S.o.println("Animal sound");
    }
}
class main {
    p s v m (S[] args) {
        Animal a = new Animal() {
            void sound() {
                S...println("Dog Barks");
            }
        };
```

```
        a.sound();
    }
}
```

o/p:- Dog Barks.

ex2: implementing an interface:

```
interface Test {
    void display();
}
class main {
    psvm(S[]args){
        Test t = new Test(){
            public void display(){
            s.o.pln("Hello");
        }
    };
    t.display();
    }
}
```

o/p — Hello

Anonymous Inner class vs Lambda expression:-

AIC

```
Runnable r = new Runnable(){
    public void run(){
        S.o.println("Running");
    }
};
```

LE:-

```
Runnable r = () -> S.o.println("running");
```

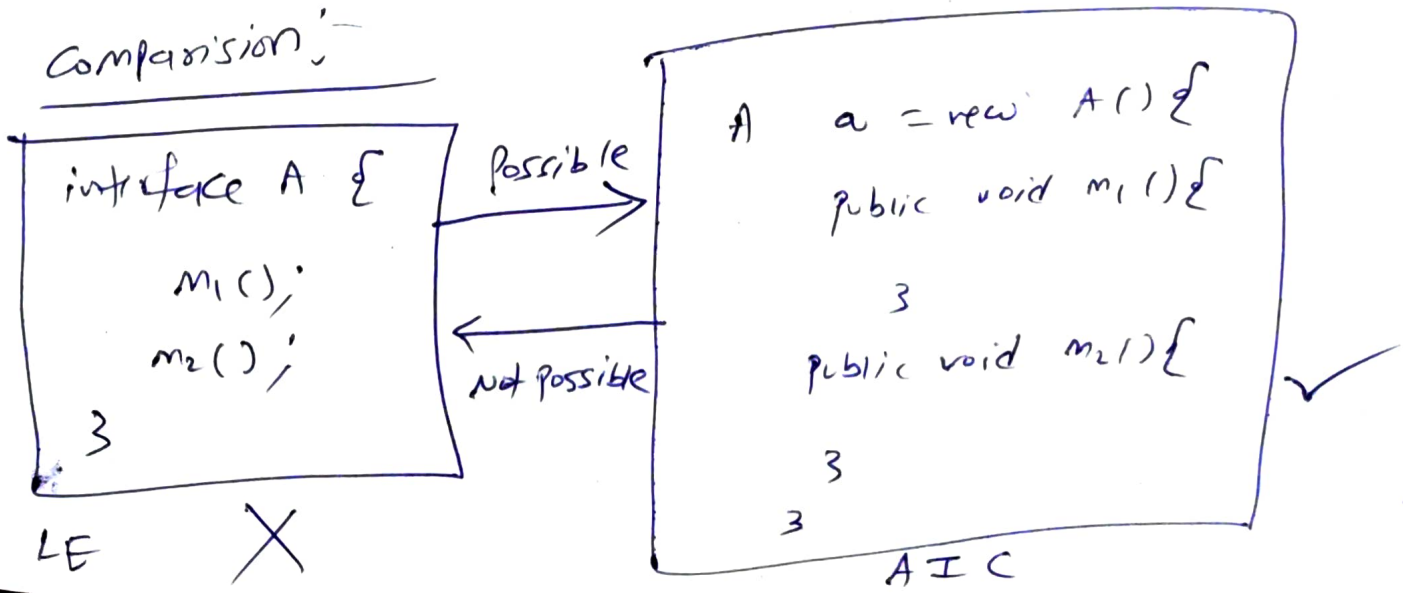
Note

Lambda can be used only with functional interfaces (interfaces having exactly only one abstract method)

why do we use Anonymous inner class:-

To provide a one-time implementation of a class or interface without creating a seperate class file, reducing code length and improving readability.

Comparision:-



NOTE: Anonymous Inner class != Lambda Expressions

If anonymous inner class implements an interface that contains single abstract method then only we can replace that anonymous inner class to Lambda expressions

Ex:

```
class AnonymousClass {
    mm {
Runnable r = new Runnable() {
    public void run() {
        s.o.pln("child Thread");
    }
};
Thread t = new Thread(r);
t.start();
s.o.pln("main Thread");
    }
}
```

Default method / virtual Extension method / Defender m:-

Default method:

without effecting implementation classes if we want to add a new method to the interface then we should go for default method.

eg:-

```
interface I {
    public void m1();
    public void m2();
    default void m3() {
        S.o.p ln ("Default method");
    }
}
```

→ if we are not satisfy this method means can we override in implemented class.

```
class Test1 implements I {
    public void m1() { }
    public void m2() { }
}
class Test2 implements I {
    public void m1() { }
    public void m2() { }
}
class Test100 implements I {
    public void m1() { }
    public void m2() { }
    public void m3() {
        S.o.p ln ("Test100 method");
    }
}
```

eg1:- satisfy with default method

```
interface Interf {
    default void m1() {
        S.o.pln ("Default method");
    }
}
class Test implements Interf {
    m m {
        Test t = new Test();
        t.m1();
    }
}
```

eg2:- not satisfy with default method:-

```
interface Interf {
    default void m1() {
        S.o.pln ("Default method");
    }
}
class Test implements Interf {
    public void m1() {
        S.o.pln ("overriding version of default
                  method");
    }
    p s v m (SL] aost) {
        Test t = new Test();
        t.m1();
    }
}
```

NOTE: Default method is not applicable in class but applicable in interface.

---

eg: by using classes not possible but possible in interfaces

```
interface Left {
    default void m1() {
        S.o.pln ("Left interface m1 method");
    }
}
interface Right {
    default void m1() {
        S.o.pln ("Right interface m1 method");
    }
}
class Test implements Left, Right {
    public void m1() {
        // S.o.pln ("our own m1 method");
        // Left.super.m1();
        Right.super.m1();
    }
    public static void main (SLT args) {
        Test t = new Test();
        t.m1();
    }
}
```

Static method :

eg:
```
interface Intef {
    public static void m1() {
        S.o.pln ("Interface static method");
    }
}
class Test implements Intef {
    ps vm (S[] args) {
        Intef.m1();  ✓
        m1();  ✗
        Test.m1();  ✗
        Test t = new Test();
        t.m1();  ✗
    }
}
```

NOTE:- Suppose everything is static then can you go through interface instead of using class because it saves memory. (without creats object).

eg:-
```
interface Intef {
    p s v m (S[] args) {
        S.o.pln ("Interface main method");
    }
}
```